بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

07 - Operations of Computer Hardware

Introduction

- Every computer must be able to perform arithmetic
- Arithmetic operations
  - Addition
  - Subtraction
- Instructions
  - add
  - sub

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

```
add a, b, c          # a gets b + c
```

    - Instructs computer to add two variables b & c and to put their sum in a

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

```
add a, b, c          # a gets b + c
```

    - Instructs computer to add two variables b & c and to put their sum in a

- All arithmetic operations have this form
- One instruction per line
- One operation is performed per instruction
- **#**: comments

Arithmetic Operations

- Notation is rigid in that each MIPS arithmetic instruction
  - Performs only one operation
  - Must always have exactly three variables
- For example, to place sum of four variables b, c, d, and e into variable a; Java code:

```
a = b + c + d + e;
```

- The following sequence of instructions adds the four variables:

Arithmetic Operations

- Compiled MIPS code:

```
add a, b, c
add a, a, d
add a, a, e
```

- It takes three instructions to sum the four variables

Operands

- Natural number of operands for an operation like addition is 3
  - Two numbers being added together and place to put sum
- Requiring every instruction to have exactly three operands
  - No more and no less
- Conforms to the philosophy of keeping HW simple
  - HW for a variable number of operands is more complicated than HW for a fixed number

Design Principle 1

- Simplicity favors regularity

- A Regularity makes implementation simpler
- Simplicity enables higher performance at lower cost

Example 1

- Java code:

```
a = b + c;
d = a - e;
```

- Compiled MIPS code:

Example 1

- Java code:

```
a = b + c;
d = a - e;
```

- Compiled MIPS code:

```
add a, b, c
```

Example 2

- Java code:

```
a = b + c;
d = a - e;
```

- Compiled MIPS code:

```
add a, b, c
sub d, a, e
```

Example 2

- Java code:

```
f = (g + h) - (i + j);
```

- Compiled MIPS code:

Example 2

- Java code:

```
f = (g + h) - (i + j);
```

- Compiled MIPS code:

```
add t0, g, h    # temp t0 = g + h
```

Example 2

- Java code:

```
f = (g + h) - (i + j);
```

- Compiled MIPS code:

```
add t0, g, h    # temp t0 = g + h
add t1, i, j    # temp t1 = i + j
```

Example 2

- Java code:

```
f = (g + h) - (i + j);
```

- Compiled MIPS code:

```
add t0, g, h    # temp t0 = g + h
add t1, i, j    # temp t1 = i + j
sub f, t0, t1   # f = t0 - t1
```